Distributed systems

Replicating data

- YouTube streams
- DropBox
- calendars on phone/web

- How do we propagate changes across systems?

	(eager) sync	(lazy) async
primary copy (master)		
no primary copy		

synchronously - when you change data in one place, you don't see "done" until changed everywhere

- async - opposite

primary copy - can only write to the primary copy

Two tasks:

- pros/cons of each quadrant
- example

Sync / primary copy

- for <u>sync</u> means have to wait until changes are received (block until them)
- <u>primary copy</u> eliminates conflicts

→ ensure that all sites see same changes, in the same order

↳ <u>"strong consistency"</u> (one of the defns)

applications: you care about ordering, and less about the speed (and accuracy)

- some banking

Con - if primary copy goes down, bad news.

async / primary copy ← single server

- e.g. streaming, google docs (on the servers)
- weather app on my phone

async = when you make a change, don't have to wait for "done" to move on

- server doesn't block until results propagate
- propagation happens "when it happens" | $\boxed{\text{eventual consistency}}$

async / no primary

- e.g. Google calendar: web/phone
  git: local / GitHub
- disadv: manage conflicts

$\boxed{\text{weak consistency}}$

eventual consistency: if updates stop coming, then eventually all nodes will have same data

sync, no primary
example: ???
-Nathaniel's game example w/ teleporting and death

How do you implement these?
How do you sync?

Dist computing community often refers to a thought experiment involving Byzantine Generals

Problem:

2 generals who want to attack a hill.

- they are camped on opposite sides of the hill.
- to succeed, they must both attack
- therefore they both have to agree
- they can send runners with messages
- the runners might get eaten by tigers

Goal: work out some kind of protocol to ensure attack only happens when both have agreed

Result: no protocol guarateed to do this with a bounded (finite) number of messages

<u>Proof</u>: Just assume you can.

Pick the smallest protocol that works, it has $\underline{P}$ messages.

Oh, suppose the $\underline{P}$th got eaten by a tiger.

Then this didn't work after all, or, we pulled it off with $P-1$ msgs (but we said this was smallest).